

Pydantic

NTP II - Hafta 9

Giriş ve Hedefler

Bu modülün temel amacı, teorik nesne yönelimli programlama (OOP) kavramlarını pratik bir veri doğrulama aracı olan Pydantic ile birleştirmektir. Temel hedefimiz, bir 'Sınıf'ın pratik bir 'Veri Modeli' olarak nasıl kullanıldığını anlamaktır.



Temel Öğrenme Hedefleri

Pydantic BaseModel kullanarak veri modeli oluşturmayı öğrenmek.
HTTP POST metodunun sunucuya veri oluşturma (kayıt ekleme) amacını anlamak.

Veri doğrulamanın (Data Validation) siber güvenlikteki hayati rolünü kavramak (GIGO prensibi).

FastAPI'nin Pydantic sayesinde otomatik oluşturduğu zengin dokümantasyonu incelemek.

OOP: Sınıf (Class) Kavramından Veri Modeline

Geleneksel OOP'de bir sınıf, objelerin yapısını ve davranışını tanımlar. Pydantic, bu teorik yapıyı pratik bir 'Veri Taşıma Nesnesi' (DTO) veya 'Şema' olarak kullanır. BaseModel'den miras almak, Python'un tip ipuçlarını (type hints) zorunlu doğrulama kurallarına dönüştürür.



HTTP POST Metodu: Veri Oluşturma

POST, genellikle yeni bir kaynak oluşturmak için kullanılır (örneğin, yeni bir kullanıcı kaydı, yeni bir ürün).
Bu metodla gönderilen veriler 'Request Body' (İstek Gövdesi) içinde taşınır.
Web API'lerinde bu gövde genellikle JSON formatındadır.



Request Body (İstek Gövdesi)

İstek gövdesi, istemcinin sunucuya kaydetmek istediği gerçek veriyi içerir. FastAPI/Pydantic ikilisi, bu JSON verisini otomatik olarak alır ve önceden tanımlanmış Python nesnelere (Pydantic modellerine) dönüştürür. Bu dönüşüm sırasında doğrulama süreci başlar.



Veri Doğrulama (Data Validation) Neden Önemli?

API'ye gelen her veri potansiyel bir tehdittir.

Doğrulama, sadece verinin formatının doğru olup olmadığını değil, aynı zamanda iş kurallarına uygun olup olmadığını kontrol eder (örneğin, bir sayının 1-5 arasında olması).

Bu işlem, API'nin istikrarını ve güvenliğini sağlar.



GIGO Prensibi: Garbage In, Garbage Out

Eğer API'ye geçersiz, kötü niyetli veya yanlış biçimlendirilmiş veri girerse, çıktı da hatalı olacaktır.

GIGO, yazılım sistemlerinin güvenilirliğini doğrudan etkileyen temel bir ilkedir. Pydantic, 'Garbage In'i daha sistem seviyesinde engellemek için tasarlanmıştır.

Güvenlik: Enjeksiyon Saldırılarından Korunma

Veri doğrulama, SQL Enjeksiyonu, XSS veya diğer enjeksiyon tabanlı saldırıları önlemede ilk savunma hattıdır.

Veri tipini ve kısıtlarını zorunlu kılarak, beklenmedik komut veya yapıların sisteme girmesi engellenir.

Özellikle string (metin) verilerinin temizlenmesi ve kısıtlanması kritiktir.



FastAPI ve Swagger UI Entegrasyonu

FastAPI, Pydantic modellerini kullanarak otomatik olarak OpenAPI (eski adıyla Swagger) dokümantasyonu oluşturur.

Bu dokümantasyon; endpoint'leri, beklenen alanları, tipleri ve zorunlulukları gösterir.

Bu sayede geliştiriciler, test araçları veya diğer sistemler için ek manuel dokümantasyon ihtiyacı azalır.



Swagger UI: Örnek JSON ve Şemalar

Pydantic modellerine eklenen açıklamalar (Field description) ve örnek veriler (json_schema_extra) doğrudan Swagger UI arayüzüne yansır. Bu, API'yi kullanan diğer geliştiricilerin (client) ne beklemesi gerektiğini hızla anlamasını sağlar.



Örnek Senaryo: Zararlı Göstergeler (IoC)

Müşteri, sisteme 'Zararlı Göstergeler' (Indicator of Compromise - IoC) eklemek için bir API uç noktası istiyor.

İstenen uç nokta: POST /indicators.

Bu senaryo, Pydantic ile modelleme ve doğrulama ihtiyacının somut bir örneğidir.

Gereksinim 1: Zararlı Gösterge Değeri

Alan adı: value.

İçerik: Zararlı verinin kendisi (örn. IP adresi: '1.2.3.4', Alan adı: 'kötücül.com', Karma değeri: 'hashdegeri...').

Kısıtlama: Bu alan kesinlikle zorunludur (non-nullable).

Gereksinim 2: Gösterge Tipi

Alan adı: type.

İçerik: Göstergenin türünü belirtir.

Kısıtlama: Sadece şu dört değerden biri olabilir: 'ip', 'domain', 'hash' veya 'url'.

Başka herhangi bir değer (örneğin, 'file' veya 'user') kesinlikle reddedilmelidir.

Gereksinim 3: Tehdit Seviyesi

Alan adı: threat_level.

İçerik: Tehdidin ciddiyetini belirten sayısal bir değer.

Kısıtlama: 1 (düşük) ile 5 (kritik) arasında bir tamsayı (integer) olmalıdır.

Varsayılan Değer: Eğer kullanıcı göndermezse, 3 (orta) olarak ayarlanmalıdır.

Hata Yönetimi Kuralı

Eğer client, tanımlanan kurallara uymayan bir veri gönderirse (örneğin, threat_level olarak 6 veya type olarak 'file'), API bu isteği kesinlikle reddetmelidir.

HTTP standardında bu tür doğrulama hataları için uygun yanıt kodu 422 (Unprocessable Entity) olarak belirlenmiştir.

FastAPI/Pydantic bu 422 yanıtını otomatik olarak oluşturur.

Çözüm: Pydantic ile OOP Bağlantısı

Müşteri talebi, kodlama açısından bakıldığında, tam olarak bir 'Sınıf' tanımıdır. Pydantic BaseModel kullanarak bu sınıfı tanımlayacağız. Bu sınıf, hem veriyi taşımak (DTO) hem de zorunlu doğrulama kurallarını uygulamak görevini görecektir.

Bu, OOP'nin 'Sınıf nedir?' sorusuna verilen en pratik yanıtlardan biridir.

Pydantic ve Python Tip İpuçları

Pydantic, Python'un yerleşik tip ipuçlarını (str, int, List, Dict) temel alır. Bu tip ipuçlarını kullanarak, çalışma zamanında (runtime) gelen verinin beklenen tipte olup olmadığını otomatik olarak kontrol eder. Bu yaklaşım, geliştirme hızını artırırken hata potansiyelini azaltır.

Pydantic Modeli Tanımlama (IoCSchema Başlangıcı)

IoCSchema sınıfını tanımlayarak Pydantic'in BaseModel sınıfından miras alıyoruz.

Miras alma (Inheritance), OOP'nin temel bir ilkesidir ve Pydantic'e model özelliklerini (doğrulama yetenekleri) kazandırır.

Bu model, verinin yapısını ve kısıtlamalarını tek bir yerde toplar.

Gelişmiş Pydantic Tipleri

Basit str veya int kullanmak yerine, Pydantic'in özel kısıtlı tiplerini kullanmalıyız.

Literal: Alanın alabileceği kesin değerleri listeler (Enum benzeri).

conint: Tamsayılar için alt ve üst limitler belirler (constrained integer).

Field: Ek metadata (açıklama, minimum uzunluk) tanımlamak için kullanılır.

Kodlama: Gerekli içe Aktarmalar (Imports)

Projemiz için gerekli temel kütüphaneleri içe aktarıyoruz:

```
'from fastapi import FastAPI, HTTPException'  
'from pydantic import BaseModel, Field, conint'  
'from typing import Literal, List, Dict, Any' (Pydantic 2.0+ uyumu için).
```

Geçici Veritabanı (In-Memory)

Gerçek bir veritabanı kullanmadan önce, verileri geçici olarak uygulamanın belleğinde (in-memory) saklamak için bir Python listesi kullanacağız.

```
'db_indicators: List[Dict[str, Any]] = []'
```

Bu yapı, IoC kayıtlarını sözlük (dictionary) formatında tutacaktır.

IoCSchema Tanımı: ID Alanı

Tanımladığımız IoCSchema, sisteme eklendikten sonra verinin nasıl görüneceğini belirtir (Response Model). Bu yüzden bir 'id: int' alanı içerir. Bu ID, verinin benzersiz tanımlayıcısıdır ve in-memory veritabanında manuel olarak oluşturulacaktır.

IoCSchema: Value Alanı Detayı

value: str = Field(..., min_length=1, description= Zararlı göstergenin değeri...1)
... (Ellipsis) kullanmak, alanın zorunlu (required) olduğunu belirtir.
min_length=1' kısıtlaması, boş bir string gönderilmesini engeller.
'description', Swagger UI'da görünecek meta veriyi sağlar.

IoCSchema: Type Alanı ve Literal

type: Literal[ip, domain, hash, url]

Literal tipini kullanarak, bu alana gelen verinin tam olarak parantez içindeki dört string değerinden biri olması gerektiğini zorunlu kılıyoruz. Hatanın otomatik olarak 422 dönmesini sağlayan en güçlü araçlardan biridir.

IoCSchema: Threat_Level ve conint

threat_level: conint(ge=1, le=5) = Field(default=3, description= Tehdit seviyesi (1-5))

conint (constrained integer) kullanılarak alt sınır (ge=1, greater than or equal to 1) ve üst sınır ('le=5', less than or equal to 5) tanımlanır.

Eğer 6 veya 0 gibi bir değer gelirse, doğrulama başarısız olur.
'default=3' ile varsayılan değer tanımlanır.

Pydantic 2.0+ Model Yapılandırması

Modelin altında tanımlanan 'Config' sınıfı, Pydantic'in çalışma şeklini yapılandırmak için kullanılır.

'json_schema_extra' sözlüğü, OpenAPI (Swagger) dokümantasyonuna eklenecek ekstra bilgileri, özellikle örnek (example) JSON verisini tanımlamamızı sağlar.



Örnek JSON Şemasının Faydası

Tanımlanan örnek ("value": "192.168.1.100", "type": "ip", "threat_level": 4), Swagger UI'da 'Example Value' olarak görünür.

Bu, API'yi test edenlerin veya entegre edenlerin ne tür bir veri beklediklerini hemen görmelerini sağlar, manuel deneme yanılma süresini kısaltır.



Ayırma Prensipleri: Giriş ve Çıkış Modelleri

API tasarımında iyi bir pratik olarak, istemciden gelen veriyi (giriş) ve sunucudan dönen veriyi (çıkış) farklı modellerle temsil ederiz. Çıkış modeli (IoCSchema) ID içerir, ancak giriş modeli (IoCCreateSchema) ID içermemelidir, çünkü ID sunucu tarafından üretilir.

IoCCreateSchema Tanımı

POST işlemi için kullanılan IoCCreateSchema, IoCSchema'nın neredeyse aynısıdır, tek farkı 'id' alanını içermemesidir. Bu model, istemcinin gönderebileceği zorunlu alanları kesin olarak tanımlar ve ID gönderme girişimlerini otomatik olarak göz ardı eder veya reddeder.



Endpoint Tanımı: POST /indicators

FastAPI'de bir POST endpoint'i tanımlıyoruz:

```
@app.post("/indicators, response_model=loCSchema, status_code=201)".
```

'response_model=loCSchema' parametresi, başarılı bir yanıtın (201 Created) loCSchema formatında olacağını garanti eder.

Parametre Olarak Pydantic Modeli

'def create_indicator(ioc: IoCCreateSchema):'
FastAPI, fonksiyon parametresindeki tip ipucunu ('ioc: IoCCreateSchema')
gördüğünde, gelen JSON gövdesini otomatik olarak bu modele göre
ayrıştırması ve doğrulaması gerektiğini anlar.
Kod, bu satırı geçtikten sonra, 'ioc' nesnesinin %100 geçerli olduğu varsayılır.

Doğrulama Başarısız Olursa...

Eğer gelen veri (JSON), 'loCCreateSchema"daki kısıtlamalara (örneğin, 'conint(le=5)') uymazsa, kod fonksiyon gövdesine bile ulaşmaz. FastAPI, otomatik olarak JSON detaylarını içeren bir 422 (Unprocessable Entity) yanıtı döndürür.



Modeli Sözlüğe Çevirme

Pydantic nesnesini, in-memory listeye ekleyebilmek için standart Python sözlüğüne (dictionary) dönüştürmemiz gerekir.
Modern Pydantic V2'de bu işlem `'.model_dump()'` metodu ile yapılır (eski versiyonda `'.dict()'` kullanılıyordu).
`'ioc_data = ioc.model_dump()'`.

Manuel ID Oluşturma Mekanizması

Gerçek bir veritabanı motoru ID'yi otomatik olarak üretirken, in-memory listemiz için bunu manuel yapmalıyız.

Yeni ID, mevcut listedeki eleman sayısının bir fazlasıdır: $\text{'new_id' = len(db_indicators) + 1}$

Bu ID, sözlüğe eklenir: $\text{'ioc_data[id] = new_id'}$.



Verinin Listeye Eklenmesi

Hazırlanan ve ID eklenen sözlük formatındaki IoC verisi, geçici veritabanı listesine eklenir: `'db_indicators.append(ioc_data)'`.
Bu adım, verinin API çalışmaya devam ettiği sürece erişilebilir olmasını sağlar.



POST Yanıtının Geri Döndürülmesi

Fonksiyon, başarıyla oluşturulan veriyi (ID dahil) geri döndürür: 'return ioc_data'.

FastAPI, bu sözlüğü alır ve bunu endpoint tanımında belirtilen 'response_model=IoCSchema"ya göre doğrular ve JSON olarak formatlar. Bu, hem giriş hem de çıkış verisinin tutarlı olmasını sağlar.

Endpoint Tanımı: GET /indicators

Sisteme eklenen tüm göstergeleri listelemek için bir GET endpoint'i tanımlanır: '@app.get(/indicators, response_model=List[IoCSchema])'. Burada response model olarak 'List[IoCSchema]' kullanılır, yani yanıt bir liste olacak ve listedeki her öge IoCSchema kurallarına uyacaktır.

GET Endpoint'inin İşleyişi

```
'def get_all_indicators(): return db_indicators'
```

Fonksiyon, in-memory listemizi doğrudan geri döndürür.

FastAPI, bu listenin içeriğini (sözlükler) alır, 'response_model'e göre doğrular ve otomatik olarak JSON formatına dönüştürerek istemciye iletir.

Pedagojik Çıkarım 1: Pydantic ve OOP

Pydantic'in BaseModel'i, teorik 'class Öğrenci:' örneğinden çok daha etkili bir şekilde OOP (Sınıf, Miras Alma, Nitelikler) kavramlarının neden yararlı olduğunu gösterir.

Öğrenciler, soyut bir kavram yerine, veri bütünlüğünü sağlayan somut bir araçla karşılaşır.

Pedagojik Çıkarım 2: API Tasarımında İyi Pratik

IoCSchema (dönüş) ve IoCCreateSchema (giriş) olarak iki ayrı model kullanmak, modern API tasarımında (özellikle CRUD operasyonlarında) yaygın ve önerilen bir yaklaşımdır.

Bu ayrım, client'ın sunucunun üretmesi gereken alanları (ID, created_at vb.) manipüle etmesini engeller.



Test Süreci: Swagger UI Kullanımı

Uygulama uvicorn ile çalışırken, '/docs' adresine gidilerek otomatik olarak oluşturulmuş Swagger UI arayüzüne erişilir.
Bu arayüz, API'yi manuel olarak cURL veya Postman kullanmaya gerek kalmadan test etme imkanı sunar.

Test Senaryosu 1: Başarılı Kayıt

Geçerli bir veri seti gönderilir (örneğin, threat_level: 4 ve geçerli bir type).
Beklenen sonuç: Sunucudan 201 (Created) HTTP durumu kodu döner.
Yanıt gövdesinde, sunucu tarafından atanan ID'yi içeren tam loCSchema verisi görülür.

Test Senaryosu 2: Geçersiz Sayısal Veri

Geçersiz bir veri gönderilir: Örneğin, 'threat_level' alanına kısıtlamanın dışında bir değer, '10' gönderilir.

Beklenen sonuç: Fonksiyonun çalıştırılması durdurulur ve 422 (Unprocessable Entity) hatası alınır.

Hata mesajı, tam olarak hangi alanın ve hangi kuralın ihlal edildiğini gösterir.

Test Senaryosu 3: Geçersiz Literal Veri

Geçersiz bir veri gönderilir: Örneğin, 'type' alanına izin verilmeyen bir değer, 'file' gönderilir.

Beklenen sonuç: Yine 422 hatası alınır.

Hata detayları, alanın sadece 'ip', 'domain', 'hash' veya 'url' değerlerinden birini kabul ettiğini belirtir.

Test Sonucu: GET ile Listeleme

Başarılı POST işlemlerinden sonra, GET /indicators endpoint'i çağrılır.
Beklenen sonuç: Daha önce eklenen tüm verilerin bir listesi JSON formatında döner.
Listenin içindeki her bir öğenin 'id' alanına sahip olduğu kontrol edilir.

Pydantic'in Arkasındaki Felsefe

Pydantic, tip güvenliğini (type safety) çalışma zamanına (runtime) taşıyarak Python'un dinamik yapısını güçlendirir.

Sadece veri modellemesi değil, aynı zamanda verinin otomatik dönüşümü (parsing) görevini de üstlenir.

Bu, modern, büyük ölçekli ve yüksek performanslı API'ler için kritik bir özelliktir.



Alan Kısıtlamalarında Detaylar: Field Metadataları

Field objesi, sadece min/max uzunluk kısıtlamaları için değil, aynı zamanda API geliştiricilerine ve kullanıcılarına yönelik zengin metadatalar eklemek için de kullanılır.

Future Pydantic kullanımlarında, 'Field' içine özel düzenli ifadeler (regex) ekleyerek, verinin formatını (örneğin e-posta veya karma formatı) daha kesin olarak doğrulayabiliriz.



Data Transfer Object (DTO) ve Mikroservis Mimarileri

[Zenginleştirme]

Pydantic modelleri, sadece tekil API'lerde değil, mikroservis mimarilerinde servisler arası iletişimi (IPC) standartlaştırmak için de DTO olarak kullanılır. API Gateway katmanında Pydantic doğrulamasının yapılması, arka uç servislere ulaşan verinin her zaman temiz olmasını garanti eder. Bu, sistem genelinde veri bütünlüğünü sağlar.



FastAPI Response Modeli: Seri Hale Getirme [Zenginleştirme]

Response model, veri doğrulaması kadar veri serileştirme (serialization) görevini de üstlenir.
API'niz bir veritabanı objesi döndürse bile, 'response_model' sadece o modelde tanımlanan alanların dışarıya sızmasını sağlar (Gizlenmesi gereken alanların korunması).

Pydantic v1 vs v2 Farkları [Zenginleştirme]

Kaynakta bahsedilen `'model_dump()'` metodu, Pydantic V2 ile gelen önemli bir değişikliktir (V1'de `'dict()'` idi).
V2, Rust tabanlı çekirdek (core) sayesinde çok daha yüksek performans sunar, bu da özellikle yüksek hacimli veri işleyen API'ler için önemlidir.

JSON Schema Standardı ve OpenAPI

Pydantic, modellerini JSON Schema formatına dönüştürebilir. JSON Schema, JSON verisinin yapısını ve kısıtlamalarını tanımlayan evrensel bir standarttır. OpenAPI (Swagger) dokümantasyonu da büyük ölçüde JSON Schema'yı temel alır, bu da Pydantic'in neden otomatik ve zengin dokümantasyon oluşturabildiğini açıklar.

Tip Kontrolünde 'typing.Literal' Kullanımının Alternatifleri

[Zenginleştirme]

Büyük veya karmaşık kısıtlı değer setleri için, 'typing.Literal' yerine Python'un standart 'Enum' (numaralandırma) sınıflarını kullanmak daha temiz bir kodlama pratiği sunar.

Pydantic, Enum sınıflarını doğal olarak destekler ve bu, kodun okunabilirliğini artırır, ancak işlevi Literal ile aynıdır.

Kısıtlı Tipler: 'conlist' ve 'constr' [Zenginleştirme]

Tamsayılarda 'conint' kullandığımız gibi, Pydantic string'ler için 'constr' (kısıtlı string) ve listeler için 'conlist' sağlar.

'constr' ile string'in minimum/maksimum uzunluğunu veya belirli bir regex desenini zorunlu kılabiliriz.

'conlist' ile ise listenin minimum/maksimum eleman sayısını belirleyebiliriz.



Hata Ayıklama ve Geliştirme Süreci

FastAPI'nin 422 yanıtları sadece hatayı bildirmekle kalmaz, aynı zamanda hatanın konumunu (field location) ve nedenini (error message) detaylı bir JSON yapısında sunar.

Bu detaylı hata mesajları, istemci tarafındaki (frontend) geliştiricilerin hızlıca entegrasyon hatalarını ayıklamasını sağlar.

Varsayılan Değerlerin Rolü

'threat_level: conint(ge=1, le=5) = Field(default=3)' tanımında olduğu gibi, varsayılan değerler API esnekliğini artırır.

İstemci bu alanı göndermezse, Pydantic otomatik olarak 3 değerini atar ve doğrulama başarılı olur.

Bu, zorunlu olmayan alanlar için veri bütünlüğünü sağlamanın pratik bir yoludur.



Özet: Pydantic'in API Gelişimine Katkısı

Pydantic, geliştirme hızını artırır (daha az el ile doğrulama kodu).
API'nin güvenliğini artırır (GIGO ve enjeksiyon önleme).
Otomatik ve güncel dokümantasyon sağlar (Swagger UI).
Bu üçlü fayda, Pydantic'i modern Python tabanlı API'ler için standart araç haline getirmiştir.



Bölüm Sonu Değerlendirmesi

Bir OOP Sınıfı neden Veri Modelleme için idealdir?
HTTP POST isteği Request Body'de veriyi nasıl taşır?
'Literal' ve 'conint' kısıtlı tiplerinin temel farkları nelerdir?
FastAPI 422 hatası hangi durumlarda tetiklenir?

Pratik Uygulama: Model Güncelleme [Zenginleştirme]

Mevcut loC'leri güncellemek için (PUT veya PATCH) yeni şemalar tasarlamamız gerekir.

PATCH (kısmi güncelleme) için, tüm alanları isteğe bağlı ('Optional' veya 'None') hale getiren modeller kullanılır.

PUT (tamamlama) için ise tüm alanlar zorunlu olmaya devam eder, ancak ID genellikle URL parametresinden alınır.

Modelde Hesaplanmış Alanlar [Zenginleştirme]

Pydantic modelleri, veritabanından gelmeyen, ancak mevcut alanlara dayanarak hesaplanan özel alanlar içerebilir.

Örneğin, 'threat_level' 5 ise 'Critical', 1-2 ise 'Low' döndüren bir 'risk_text' alanı tanımlanabilir.

Bu, iş mantığının bir kısmını modelin içine taşımayı sağlar.

FastAPI Dependency Injection (Bağımlılık Enjeksiyonu)

[Zenginleştirme]

Pydantic modelinin otomatik doğrulama için kullanılması, FastAPI'nin Bağımlılık Enjeksiyonu (DI) sisteminin bir yan ürünüdür. DI, veritabanı bağlantıları, güvenlik tokenları veya Pydantic modelleri gibi bileşenleri fonksiyonlara otomatik olarak sağlamak için kullanılır.

Siber Güvenlikte IoC Türlerinin Önemi

IP adresi, domain ve hash gibi farklı IoC türlerini ayırmak ('ip', 'domain', 'hash', 'url'), güvenlik sistemlerinin bu verilere farklı tepkiler vermesini sağlar. Örneğin, bir 'ip' IoC'si güvenlik duvarında engellenirken, bir 'hash' değeri dosya sisteminde kontrol edilir. Literal kısıtlaması bu ayrımı zorunlu kılar.

Veri Bütünlüğü ve Tutarlılık

Geleneksel olarak, veri bütünlüğü kısıtlamaları veritabanı şemalarında (CHECK kısıtlamaları) tanımlanırdı.

Pydantic, bu kısıtlamaları uygulama katmanına (API katmanı) taşıyarak veritabanına ulaşmadan önce hatalı verinin elenmesini sağlar.

Bu, veritabanı üzerindeki yükü azaltır ve hata yanıtlarını hızlandırır.

Kısıtlamaların Genişletilmesi: 'validator' ve 'model_validator' [Zenginleştirme]

Pydantic, sadece tek bir alana uygulanan basit kısıtlamaların ötesine geçer. Özel 'validator' fonksiyonları ile iki veya daha fazla alan arasındaki mantıksal ilişkiyi doğrulayabiliriz. Örneğin, 'eğer type = ip ise, value alanının IPv4 formatında olması zorunludur' gibi kurallar eklenebilir.



Modelde Özel Hata Mesajları [Zenginleştirme]

Pydantic'in otomatik 422 yanıtları genellikle teknik detaylar içerir. Ancak, kullanıcı deneyimini artırmak için, geliştiriciler Pydantic'in hata işleme mekanizmasını genişleterek daha anlaşılır ve kullanıcı dostu özel hata mesajları tanımlayabilir.

Pydantic ve Tescilli Tipler [Zenginleştirme]

Pydantic, Python'un standart tipleri dışında, yaygın olarak kullanılan tescilli formatlar için özel tipler sunar (örn: EmailStr, HttpUrl, UUID). Bu tipler kullanıldığında, Pydantic otomatik olarak string değerinin standart bir e-posta veya URL formatına uyup uymadığını kontrol eder.



API Geliştirme Sürecine Etkisi

Pydantic ve FastAPI, 'Kontrat Odaklı Geliştirme' (Contract-First Development) yaklaşımını destekler.

API'nin 'kontratu' (yani, Pydantic modelleri ve OpenAPI şeması) kodlama başlamadan önce bile netleşir.

Bu, ön uç ve arka uç ekiplerinin paralel çalışmasını kolaylaştırır.



Gelecek Konular: Veritabanı Entegrasyonu [Zenginleştirme]

Bu hafta in-memory listeler kullandık. Ancak gerçek uygulamalarda veriler kalıcı olarak veritabanında saklanır.

Sonraki aşamalarda, SQLAlchemy gibi bir ORM (Object-Relational Mapper) ile Pydantic modellerinin nasıl entegre edildiğini göreceğiz.

Pydantic, ORM objelerini alıp dışarıya güvenli JSON olarak dönüştürme görevini sürdürür.



Güvenlik Özeti: Temel Savunma Mekanizmaları

1. Tip Kontrolü: Beklenmeyen veri türlerini reddetme.
 2. Kısıtlama Kontrolü: Sayısal aralıkları (1-5) ve uzunlukları zorunlu kılma.
 3. Enumerasyon Kontrolü: Sadece izin verilen string değerlerini kabul etme ('ip', 'domain', vb.).
- Bu katmanlı yaklaşım, API'nin sağlamlığını garanti eder.

Kritik Kavramlar: BaseModel ve Veri Bütünlüğü

Pydantic BaseModel, veri bütünlüğünü (data integrity) API giriş noktasında sağlamak için modern ve etkili bir araçtır. Bu sayede, veritabanı veya iş mantığı katmanları, zaten temizlenmiş ve doğrulanmış veriyle çalışır durumdadır.



Soru & Cevap

Bu modülde öğrendiğimiz Pydantic ve FastAPI entegrasyonu hakkında sorular. Önümüzdeki hafta, oluşturulan bu API yapısının test otomasyonuna (Unit Testing) nasıl dahil edileceği incelenecektir.



Pydantic

Süleyman **EZDEMİR**

suleyman.ezdemir@giresun.edu.tr

